

Intelligent HR Equipment Support Assistant Using LangChain and LangGraph

Domain: Multi-Agent Systems, LangChain, LangGraph, Tools, Memory, SQLite, HITL, MCP

Environment: Python 3.11 | Visual Studio Code | Streamlit | SQLite3

1. Project Overview

Project Title

Intelligent HR Equipment Support Assistant Using LangChain and LangGraph

Problem Statement

Employees frequently encounter issues related to laptops, software installations, office Wi-Fi connectivity, passwords, system access permissions, printers, and peripherals. In traditional corporate setups, employees must contact HR or IT helpdesk representatives for every issue—even routine, well-documented inquiries that follow clear standard operating procedures (SOPs). This manual process introduces severe delays, queue backlogs, and unnecessary workload for support personnel. Furthermore, conventional rule-based chatbots provide only rigid, predetermined responses without context, cannot maintain state across multi-turn queries, cannot query live databases, and lack safeguards for sensitive privileged operations.

Brief Description of the Project

The **Intelligent HR Equipment Support Assistant** is an enterprise-grade Agentic AI application designed to autonomously triage, troubleshoot, and resolve employee equipment and HR issues. The assistant receives natural-language user queries (e.g., *'My laptop is not turning on'*, *'The payroll application crashes whenever I open it'*, or *'I forgot my portal password'*), sanitizes user input via Pydantic, retrieves short-term conversational context and long-term employee memory from SQLite, classifies the issue into four primary functional categories (Hardware, Technical, Access, General HR), and routes it across specialized agents orchestrated via a compiled LangGraph state graph. The system executes operational tools for ticket generation and status checks, and halts autonomous execution to require Human-in-the-Loop (HITL) approval whenever credential resets or expensive asset purchases are requested.

2. Objectives & Proposed Solution

Project Objectives

- Build an end-to-end, runnable Agentic AI application for corporate HR and IT equipment support.
- Implement autonomous query classification with $\geq 90\%$ accuracy using hybrid LLM and pattern matching.
- Orchestrate multi-agent control flow using a cyclic, compiled LangGraph StateGraph.
- Implement operational LangChain tools for ticket creation, status checks, safe AST calculations, and SOP searches.
- Maintain dual-layer memory: short-term multi-turn conversation logs and long-term SQLite employee profiles.
- Enforce Human-in-the-Loop (HITL) governance for sensitive actions (passwords, system access, high-value assets).
- Deliver a dual interface: visual Streamlit web dashboard and interactive Rich terminal CLI.
- Ensure secure coding standards: strict Pydantic input validation, AST evaluation, and zero-eval execution.

How the Agentic AI Solution Works

The system separates decision-making (the LLM brain and classifier) from operational execution (tools and database drivers). The LangGraph StateGraph acts as the central router managing 8 discrete nodes: `classify_query`, four specialized agents (`hardware_agent`, `technical_agent`, `access_agent`, `general_agent`), `create_ticket`, `human_approval`, and `final_response`. Memory from SQLite provides continuity across turns, alerting agents to prior software crashes or departmental hardware configurations.

Key Features

- **Hybrid Intent Classification:** Instant zero-latency regex word-boundary matching with local Ollama/Granite LLM fallback.
- **4 Specialized Domain Agents:** Tailored diagnostic logic for hardware, software, security access, and general HR.
- **Dynamic Tool Calling:** Creates unique TKT-XXXXXXX support tickets and queries statuses in real-time.
- **Persistent SQLite Memory:** Preserves employee history and correlations (e.g. repeated crashes escalate priority).
- **Human-in-the-Loop Security Escrow:** Halts autonomous execution and creates APP-XXXXXX approval tokens.
- **Safe AST Calculator:** Evaluates math expressions without using dangerous Python `eval()`.
- **Interactive Streamlit UI:** Complete with colored agent routing badges, approval simulation buttons, and ticket inspector.

3. Implementation & Results

Technologies and Tools Used

Component	Technology	Version	Role in Architecture
Programming Language	Python	3.11.9	Core implementation runtime
Agent Orchestration	LangGraph	1.2.11	StateGraph, cyclic edges, conditional routing
LLM Framework	LangChain Core	1.6.3	Tool registry, prompts, model bindings
Data Validation	Pydantic	2.13.5	Input sanitization, schemas, enums
Relational Database	SQLite3	Native	Tickets, profiles, conversation history
Web Application UI	Streamlit	1.56.0	Chat interface, memory viewer, approval buttons
Terminal Interface	Rich	14.3.3	Colorized terminal panels and CLI tables
Local Model Backend	Ollama / Granite	granite3-de nse	Local semantic classification engine

Working Process

The workflow follows 12 distinct steps: (1) Employee submits natural-language query; (2) Pydantic sanitizes input; (3) SQLite retrieves long-term profile and recent conversation turns; (4) `SupportState` is initialized; (5) `classify_query` evaluates intent and urgency; (6) `LangGraph route_query` routes to the specialist agent; (7) Agent queries SOP knowledge base; (8) Agent checks for sensitive/high-cost criteria; (9) Conditional edge sends state to `human_approval`, `create_ticket`, or `final_response`; (10) Tools execute actions; (11) Turn logs and memory are saved to SQLite; (12) User receives structured markdown response.

Screenshots / Output Representations

```
Sample Terminal Output: Hardware Issue Resolution
> [EMP1024] My laptop is not turning on.
[Category]: HARDWARE | [Agent]: Hardware Agent | [Priority]: HIGH
Hello EMP1024, (Department: Finance)
### Laptop Power & Boot Diagnostics:
1. Connect original OEM charger and verify LED indicator.
2. Perform 30-second hard reset.
--- [Ticket Created]: TKT-412C85A3 | Status: Open | Priority: High

Sample HITL Approval Output: Password Reset Security
> [EMP1024] I forgot my employee portal password.
[Notice]: Human-in-the-loop approval required for credential reset.
[Approval Request Created]: APP-3A96B0 | Action: ACCESS_SENSITIVE_ACTION
Current Status: Pending Human Administrator Verification
```

Results Achieved

The system was evaluated against 12 comprehensive test scenarios covering all required domains. **100.00% classification accuracy** and **zero routing failures** were achieved with an average latency of 0.0803 seconds.

#	Query Scenario	Predicted	Agent	Tool Executed	Approval?	Result
1	My laptop is not turning on.	hardware	hardware_agent	create_ticket	No	PASS
2	Office printer is not printing...	hardware	hardware_agent	create_ticket	No	PASS
3	Payroll app crashes on open.	technical	technical_agent	create_ticket	No	PASS
4	Office Wi-Fi not connecting.	technical	technical_agent	create_ticket	No	PASS
5	Forgot employee portal password.	access	access_agent	human_approval	Yes	PASS
6	Need access to attendance system.	access	access_agent	human_approval	Yes	PASS
7	How can I apply for leave?	general	general_agent	troubleshoot_sop	No	PASS
8	Status of ticket TKT-12345678?	general	general_agent	check_ticket	No	PASS
9	What is the meaning of life?	general	general_agent	troubleshoot_sop	No	PASS
10	Need a new laptop, mine is broken.	hardware	hardware_agent	human_approval	Yes	PASS
11	Monitor screen is blank/flickering.	hardware	hardware_agent	human_approval	Yes	PASS
12	Working hours & shift timings?	general	general_agent	troubleshoot_sop	No	PASS

4. Conclusion & Future Scope

Project Conclusion

The Intelligent HR Equipment Support Assistant validates the power of Agentic AI workflows over basic chatbots. By decomposing support operations into decision-making models, operational tools, persistent SQLite memory, and explicit LangGraph state transitions, the system resolves routine inquiries autonomously while enforcing strict human oversight over credential and financial actions. The project represents a complete, secure, and production-ready enterprise solution.

Challenges Faced

- **LangChain 1.4 StructuredTool Invocation:** Decorated tools could not be called directly as functions; solved by dual-exporting native Python functions.
- **Keyword Substring Collisions:** 'apply' matched 'app' and 'laptop' matched 'pto'; resolved with regex word boundary (b) logic.

- **Ollama Daemon Timeouts:** Offline socket hanging was eliminated using a 0.2s socket liveness probe.
- **Windows Terminal Encoding:** PowerShell cp1252 unicode crashes were resolved with UTF-8 sys.stdout wrappers.
- **Safe AST Math Evaluation:** Completely prevented arbitrary code execution by avoiding eval().

Future Enhancements

- Direct IBM Cloud watsonx.ai foundation model integration (IBM Granite 3.0).
- Retrieval-Augmented Generation (RAG) over corporate HR manuals using ChromaDB.
- Two-way REST API synchronization with ServiceNow and Jira Service Management.
- Multilingual translation (Tamil, English, Hindi) and voice assistance via Whisper API.

References

1. IBM Agentic AI Internship curriculum video lectures, notes, and transcripts.
2. IBM watsonx.ai documentation: <https://www.ibm.com/products/watsonx-ai>
3. LangChain Documentation: <https://python.langchain.com/>
4. LangGraph Documentation: <https://langchain-ai.github.io/langgraph/>
5. Pydantic Data Validation: <https://docs.pydantic.dev/>
6. Model Context Protocol (MCP) Specification: <https://modelcontextprotocol.io/>